

# set\_wifi

REFERENCE

API consumer

create or modify a Wi-Fi profile on the reader

## Overview

`set_wifi` writes a Wi-Fi profile to the reader — it **creates** a new saved profile or **modifies** an existing one (selected by `wifiConfig.operation` = `CREATE` / `MODIFY`). One payload configures the interface, the access point (ESSID, preference), the security type and its credentials (WPA2/WPA3 Personal or Enterprise), and DHCP IPv4. It is a state-changing command on the device-management ([MGMT](#)) endpoint. The response is status-only — it echoes **no configuration and no credentials**; confirm the result with `get_wifi`.

`set_wifi` **is state-changing**: `accessPoint.connect: true` **switches networks and can sever the MQTT control session**. Use `connect: false` to save a profile without switching. See the warning callout in [Notes and Caveats](#).

## Details

| Property                | Value                                                                                                                                                                                           |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Interaction Pattern     | Request / Response (app publishes a command; device publishes the correlated reply)                                                                                                             |
| State Change            | STATE-CHANGING (writes a saved profile; may switch the active connection)                                                                                                                       |
| Reversibility           | REVERSIBLE (remove a created profile with <code>delete_wifi_profile</code> ; re-send <code>set_wifi</code> <code>MODIFY</code> to change settings)                                              |
| Supported Operations    | <code>CREATE</code> (new profile), <code>MODIFY</code> (update existing, matched by <code>ssid</code> )                                                                                         |
| Required Request Fields | <code>command</code> , <code>requestId</code> , <code>wifiConfig</code> (with <code>operation</code> and <code>accessPoint</code> ; inside it, <code>ssid</code> and <code>isPreferred</code> ) |
| Applies To              | <b>Endpoint:</b> Management (MGMT). <b>Reader:</b> RFD40 series, RFD90 series                                                                                                                   |
| API Version             | <code>v1.1</code> ( <a href="#">API Version Reporting</a> )                                                                                                                                     |
| Related                 | <code>get_wifi</code> , <code>delete_wifi_profile</code> , <code>install_certificate</code>                                                                                                     |

## When to Use

Use `set_wifi` to **provision or update the reader's Wi-Fi** — during onboarding, when a network changes, or to mark a profile preferred.

- `CREATE`: first-time provisioning of a new ESSID. Fails with [code 18](#) if the ESSID already exists.
- `MODIFY`: update an existing profile (password, security, preference), matched by `ssid`. Fails with [code 15](#) if the ESSID is not found. A `MODIFY` can be a partial payload — e.g. just `operation` + `accessPoint.{isPreferred, ssid}` to set the preferred flag.
- Pre-provision safely**: send `connect: false` to save a profile without switching the active connection — the recommended pattern when managing a reader over the same Wi-Fi that carries the [MDM](#) session.

## Topic Mapping

| Message  | Direction    | Topic                                                          | Note                                                                                                |
|----------|--------------|----------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| Request  | App → Reader | <code>&lt;tenantId&gt;/MGMT/clients/cmnd/&lt;serial&gt;</code> | On the management endpoint.                                                                         |
| Response | Reader → App | <code>&lt;tenantId&gt;/MGMT/clients/resp/&lt;serial&gt;</code> | Correlated reply to the request above; the in-payload <code>requestId</code> links the twin topics. |

Topics follow the pattern `<tenantId>/<topic>/<serial>` — the tenant id assigned to you at provisioning (a slot you fill in, never a literal), the base topic you configure on the endpoint, and the reader's serial number, which the reader appends itself in its model-prefixed form (for example `RFD40-212735201D0053`). `App → Reader` means your app publishes and the reader subscribes; `Reader → App` means the reader publishes and your app subscribes.

Credentials are presented once at `CONNECT`. The `requestId` you send is echoed in the response so you can pair reply to request. The endpoint's `mqttParams` govern QoS and retain, not this command.

See [Topics and Channels](#).

## Key Fields

- `wifiConfig.operation` (request) — `CREATE` or `MODIFY`; determines whether a new profile is added or an existing one (matched by `ssid`) is updated.

- `wifiConfig.accessPoint.essid` (request) — the network name; the create target / modify key (case-sensitive).
- `wifiConfig.accessPoint.connect` (request) — ⚠️ `true` switches the active connection (can sever the MDM session); `false` just saves. See the notes below.
- `wifiConfig.accessPoint.security.securityType` (request) — selects which `securityDetails` key you populate (WPA2/WPA3 Personal or Enterprise).
- `response.code` (response) — read this to determine success vs failure (see [Response Codes](#)).

## Request Payload

The request envelope is `command`, `requestId`, and the `wifiConfig` payload. Use the **Schema** tab for the full field reference and the **Example** tab for a ready-to-send WPA2-Personal `CREATE` (enterprise variants are in [Examples](#)).

Credentials in the examples are the contract's sample values, not real secrets — passwords, passphrases, and identities are write-only (see [Request Semantics](#)).

Pick a **profile scenario** below; the Example and Schema update together. Use the **Schema** tab for the full field reference and the **Example** tab for the concrete payload.

Profile scenario

Create WPA2 personal



## Request payload

Example: WPA2-Personal CREATE

```
1  {
2    "command" : "set_wifi" ,
3    "requestId" : "abc123" ,
4    "wifiConfig" : {
5      "interfaceName" : "wlan0" ,
6      "enableInterface" : true ,
7      "operation" : "CREATE" ,
8      "accessPoint" : {
9        "connect" : false ,
10       "isPreferred" : false ,
11       "autoConn" : true ,
12       "essid" : "TestAP1" ,
13       "enableSecurity" : true ,
14       "security" : {
15         "securityType" : "WPA2Personal" ,
16         "securityDetails" : {
17           "WPA2Personal" : {
18             "password" : "test@123"
19           }
20         }
21       }
22     }
23   }
24 }
```

## Request payload

\* required • — not applicable • Colored badge = JSON type

| # | FIELD         | TYPE   | ENUM / UNITS | EXAMPLE |
|---|---------------|--------|--------------|---------|
| 1 | command *     | string | —            | set_v   |
| 2 | requestId *   | string | —            | abc12   |
| 3 | wifiConfig *  | object | —            | —       |
| 4 | interfaceName | string | —            | wlan0   |

| #  | FIELD               | TYPE    | ENUM / UNITS  | EXAMP |
|----|---------------------|---------|---------------|-------|
| 5  | enableInterfac<br>e | boolean | —             | true  |
| 6  | operation *         | enum    | CREATE MODIFY | CREAT |
| 7  | accessPoint *       | object  | —             | —     |
| 8  | connect             | boolean | —             | false |
| 9  | isPreferre<br>d *   | boolean | —             | false |
| 10 | autoConn            | boolean | —             | true  |
| 11 | essid *             | string  | —             | ZWire |
| 12 | enableSecur<br>ity  | boolean | —             | true  |

| #  | FIELD                 | TYPE    | ENUM / UNITS                                               | EXAMP  |
|----|-----------------------|---------|------------------------------------------------------------|--------|
| 13 | ipv4Configu<br>ration | object  | —                                                          | —      |
| 14 | ipAddress             | string  | ipv4                                                       | 192.1  |
| 15 | subnetMas<br>k        | string  | ipv4                                                       | 255.2  |
| 16 | gateway               | string  | ipv4                                                       | 192.1  |
| 17 | enableDhc<br>p        | boolean | —                                                          | true   |
| 18 | dnsServer             | string  | ipv4                                                       | 192.1  |
| 19 | domainNam<br>e        | string  | —                                                          | exampl |
| 20 | security              | object  | —                                                          | —      |
| 21 | securityType *        | enum    | WPA2Personal WPA3Personal<br>WPA2Enterprise WPA3Enterprise | WPA2P  |
| 22 | securityDetail *<br>s | object  | —                                                          | —      |
| 23 | WPA2Pe<br>rsonal      | object  | —                                                          | —      |

| #  | FIELD               | TYPE   | ENUM / UNITS  | EXAMP    |
|----|---------------------|--------|---------------|----------|
| 24 | password            | string | —             | test()   |
| 25 | WPA3Personal        | object | —             | —        |
| 26 | password            | string | —             | —        |
| 27 | WPA2Enterprise      | object | —             | —        |
| 28 | authentication      | enum   | tls ttls peap | peap     |
| 29 | innerAuthentication | enum   | none mschapv2 | mschapv2 |
| 30 | identity            | string | —             | test     |
| 31 | anonymousIdentity   | string | —             | test     |

| #  | FIELD            | TYPE   | ENUM / UNITS                                        | EXAMP |
|----|------------------|--------|-----------------------------------------------------|-------|
| 32 | password         | string | —                                                   | test( |
| 33 | phrase           | string | —                                                   | test( |
| 34 | certificate      | array  | —                                                   | —     |
| 35 | key              | enum   | ca_cert client_key<br>client_cert cert_key_password | ca_ce |
| 36 | value            | string | —                                                   | wifi_ |
| 37 | protocol         | enum   | WPA2_Enterprise_CCMP                                | WPA2_ |
| 38 | WPA3Enterprise   | object | —                                                   | —     |
| 39 | authentication * | enum   | tls ttls peap                                       | peap  |



| #  | FIELD               | TYPE   | ENUM / UNITS                                     | EXAMP |
|----|---------------------|--------|--------------------------------------------------|-------|
| 40 | innerAuthentication | enum   | none mschapv2                                    | mscha |
| 41 | identity            | string | —                                                | test  |
| 42 | anonymousIdentity   | string | —                                                | test  |
| 43 | password            | string | —                                                | test  |
| 44 | phrase              | string | —                                                | test  |
| 45 | certificate         | array  | —                                                | —     |
| 46 | key                 | enum   | ca_cert client_key client_cert cert_key_password | ca_ce |
| 47 | value               | string | —                                                | wifi_ |

| #  | FIELD   | TYPE | ENUM / UNITS                                                                                                                                           | EXAMP |
|----|---------|------|--------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| 48 | profile | enum | WPA3_Enterprise_CCMP<br>WPA3_Enterprise_CCMP_256<br>WPA3_Enterprise_GCMP_128<br>WPA3_Enterprise_GCMP_256_SHA256<br>WPA3_Enterprise_GCMP_256_SUITEB_192 | WPA3_ |

## Request Semantics

- **CREATE vs MODIFY.** `CREATE` adds a new profile (fails `18` if the `ssid` exists); `MODIFY` updates an existing profile matched by `ssid` (fails `15` if not found). The device enforces a maximum number of saved profiles — exceeding it returns `19`; delete unused profiles with `delete_wifi_profile`. For Enterprise TLS, the `ca_cert` / `client_cert` / `client_key` referenced in `certificate` must already be installed via `install_certificate`.
- **Credentials are write-only — mask them.** Supply passwords, passphrases, and identities in the request (the examples carry sample values); the device never echoes them — `set_wifi`'s response carries no credentials, and `get_wifi` returns stored secrets masked.
- **Enterprise inner authentication.** For `ttls` and `peap`, `innerAuthentication` must be `mschapv2` and defaults to `mschapv2` when omitted; for `tls` it is not applicable and can be omitted, as the `tls` variants on this page do.
- **IPv4 only, DHCP only.** In the current API version, `set_wifi` supports **IPv4 with DHCP assignment only**. The `ipv4Configuration` static fields ( `ipAddress` / `subnetMask` / `gateway` / `dnsServer` ) are modeled in the schema but **not supported yet** (planned for a future release), and **IPv6 is not available**. Use `enableDhcp: true`.

## Response Payload

`set_wifi` returns a **status-only** envelope — `command`, `requestId`, `apiVersion`, and `response`. There is no configuration echo and no credential in the response; to confirm the saved profile, follow up with `get_wifi`.

## Response payload

```

1  {
2    "command" : "set_wifi" ,
3    "requestId" : "abc123" ,
4    "apiVersion" : "V1.1" ,
5    "response" : {
6      "code" : 0 ,
7      "description" : "Success"
8    }
9  }

```

## Response payload

\* required   • — not applicable   • Colored badge = JSON type

| # | FIELD         | TYPE    | ENUM / UNITS               | EXAMPLE  | DESCRIPTION                                                                                        |
|---|---------------|---------|----------------------------|----------|----------------------------------------------------------------------------------------------------|
| 1 | command *     | string  | —                          | set_wifi | Echoes the executed command.                                                                       |
| 2 | requestId *   | string  | —                          | abc123   | Echoes the request's requestId — pairs reply to request.                                           |
| 3 | apiVersion *  | enum    | V1.0 V1.1 V1.2 V1.21 V1.22 | V1.1     | API version reported by this command. Emitted per-handler, not device-wide.                        |
| 4 | response *    | object  | —                          | —        | Status-only result — set_wifi echoes no configuration and no credentials.                          |
| 5 | code *        | integer | 0-101                      | 0        | 0 = Success; non-zero = error. Required by the contract. See <a href="#">Response Codes</a> below. |
| 6 | description * | string  | —                          | Success  | Human-readable status.                                                                             |

# Response Codes

`set_wifi` returns `response.code`. The codes applicable to this command are below; for the full table see the [Response Codes](#) and [Correlation and Request/Response Pairing](#).

| Code | Description                                                                                                                                                                                                           |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0    | Success — the profile was created or modified.                                                                                                                                                                        |
| 2    | Invalid payload ( <code>IOT_ERROR_INVALID_PAYLOAD</code> ) — the JSON is malformed or has invalid field values. Validate against the schema and correct.                                                              |
| 15   | WIFI Error - SSID not found ( <code>IOT_ERROR_SSID_NOT_FOUND</code> ) — a <code>MODIFY</code> targeted an <code>essid</code> that is not saved. Verify the name or <code>CREATE</code> it first.                      |
| 17   | WIFI Error - SSID missed ( <code>IOT_ERROR_SSID_MISSED</code> ) — the required <code>essid</code> field is missing from the payload.                                                                                  |
| 18   | WIFI Error - SSID already exist ( <code>IOT_ERROR_SSID_ALREADY_EXIST</code> ) — a <code>CREATE</code> used an <code>essid</code> that already exists. Use <code>MODIFY</code> , or delete the existing profile first. |
| 19   | WIFI Error - SSID count overflow ( <code>IOT_ERROR_SSID_LIMIT_OVERFLOW</code> ) — the saved-profile limit is reached. Delete an unused profile with <a href="#">delete_wifi_profile</a> .                             |
| 20   | Wifi is not supported ( <code>IOT_ERROR_WIFI_INTER_NOT_SUPPORTED</code> ) — the device has no Wi-Fi interface; use Ethernet.                                                                                          |
| 23   | Invalid enum value ( <code>IOT_ERROR_INVALID_ENUM</code> ) — a field (e.g. <code>operation</code> , <code>securityType</code> , <code>authentication</code> ) is outside its allowed set.                             |
| 101  | Default error — generic catch-all.                                                                                                                                                                                    |

## Examples

Publish a `set_wifi` request; the device replies with the status-only envelope shown under [Response Payload](#). Every contract example is in the **Profile scenario** dropdown under [Request Payload](#):

- **Create WPA2 personal and Modify WPA2 personal** — a pre-shared key.
- **Set preferred AP** — a partial `MODIFY` that needs only `operation` plus `accessPoint`.  
`{isPreferred, essid}`.
- **Create WPA2 enterprise TLS / TTLS / PEAP and Create WPA3 enterprise TLS / TTLS / PEAP** — installed certificates referenced as `{key, value}`, an EAP `authentication` method and a `protocol`; the TTLS and PEAP variants add `innerAuthentication`.

## Notes and Caveats



`connect: true` CAN SEVER THE CONTROL SESSION

`accessPoint.connect: true` disconnects the currently connected profile and connects to the target ESSID. If the reader manages itself over the same Wi-Fi that carries its MQTT/MDM session, switching to a different network **drops the control channel** and can strand the device. When managing a reader over Wi-Fi, use `connect: false` (save without switching), and undo an unwanted profile with `delete_wifi_profile` rather than connecting onto it.

Because the response is status-only, **verify the result with** `get_wifi` — check that the profile was saved (and its `isPreferred` / `accessPoint.status`). For the concept-level view, see [Network Configuration](#).

**Related:** [Network Configuration](#) · `get_wifi` · `delete_wifi_profile` · `install_certificate` · [Response Codes](#)